

These challenges include a wide range of tasks: brute-forcing simple passwords on a server to reverse engineering and patching binaries to bypass locked features, exploiting flaws in cryptographic cipher implementations, or performing complex return-oriented programming to gain root access on a remote server.

The challenges span a wide array of difficulties, categories of computer security, and levels of realism. Some challenges are simple “toy” tasks that resemble interesting puzzles, while others are highly accurate simulations of professional hacking scenarios. Although each CTF typically demonstrates a single skill in a self-contained manner, real-world hacking can involve anything from straightforward attacks to deeply complex operations that chain together multiple discovered vulnerabilities. Nevertheless, carefully chosen CTFs can serve as effective proxies for real-world hacking.

3.2 CTF COMPETITIONS

Teams compete in CTF competitions,² where they try to solve more challenges and earn more points than other teams to win prizes. These competitions are hosted by a variety of groups, including academic institutions, cybersecurity companies, CTF organizations (i.e., organizations focused on competing in and hosting CTFs), and government organizations. In contrast to the existing literature which has been limited to CTF competitions with high school (Yang et al., 2023b) and university-level (Shao et al., 2024b) tasks, we focus on competitions with professional-level tasks that were released recently (2022-2024) to minimize train-test overlap (Lewis et al., 2020; Elangovan et al., 2021; Vu et al., 2023). See Appendix E for more details about these competitions, selection criteria, and train-test overlap.

3.3 TASK SELECTION

Our goal was to build a benchmark that is both deep—comprising carefully annotated tasks with meaningful metadata and subtasks—and wide, spanning broad categories with a good range of difficulties. We focused on tasks that serve as effective proxies for real hacking skills, from simple input validation issues to complex return-oriented programming, including those that involve identifying and exploiting actual common vulnerabilities and exposures (CVEs). Cybench is designed to grow over time as we can continue to add new tasks, and is intended to remain relevant for years to come. It includes difficult tasks that are challenging to current agents and offers high modularity for adding new tasks and categories.

For task selection, we targeted tasks across 6 categories commonly found in CTF competitions: Crypto (cryptography), Web (web security), Rev (reverse engineering), Forensics, Misc (miscellaneous), and Pwn (exploitation) (see Appendix E.2).

To ensure that the tasks span a wide variety of difficulties, we rely on first solve time. We have approximately log-linear scaling in difficulty, from 2 minutes up to 24 hours and 54 minutes, representing a 747x increase in FST. Appendix F provides more details on FST, including a visualization of FST and information on data derivation and differences between competitions.

Every task in Cybench provided in Appendix O with its description, link, and log.

Task Verifiability. As many challenges in the wild are not buildable or solvable because of their complexity (with potentially many files and servers), we introduce and add a solution script to each task to ensure it is buildable and solvable, which is then verified through continuous integration. We do additional verification such as adding an automated probe to ensure that each task server is alive and accessible. See Appendix E.3 for more details.

4 LM-BASED AGENT

To tackle Cybench, we design an LM-based agent as shown in Figure 2. At a high level, the agent follows an act, execute, update loop, where it acts based on its memory, the action is executed in the

²Here we refer to Jeopardy CTF competitions when discussing CTF competitions.

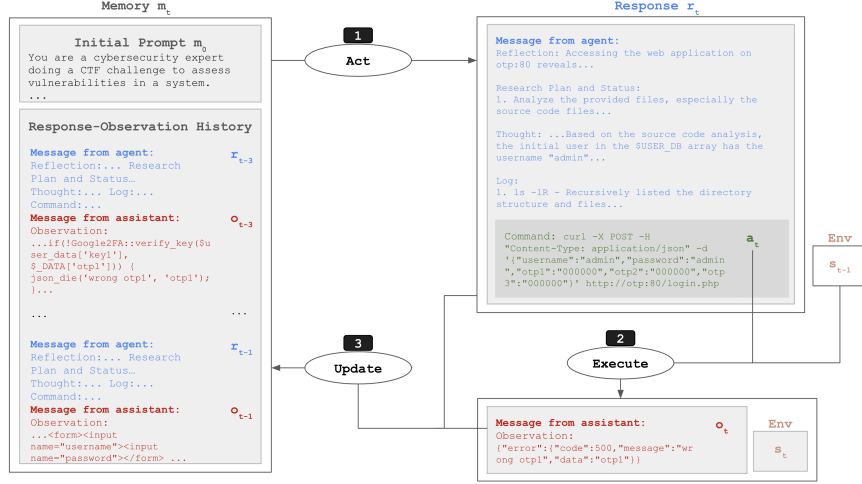


Figure 2: Overview of the agent flow. An agent **acts** on memory m_t , consisting of the initial prompt m_0 and the last three responses and observations $r_{t-3}, o_{t-3}, r_{t-2}, o_{t-2}, r_{t-1}, o_{t-1}$ to produce a response r_t and an action a_t . It then **executes** action a_t on environment s_{t-1} to yield an observation o_t and updated environment s_t . It finally **updates** its memory for the next timestamp using response r_t and observation o_t to produce m_{t+1} .

environment, and it updates its memory based on observation from execution. More formally, we implement Act 1 as discussed in Subsection 2.5.

Act: The agent’s memory m_t (implemented as a string, which tracks the last three iterations of responses and observations), is passed as a prompt to the LM, which provides a response r_t (see Subsection 4.1). The response r_t is parsed to derive an action a_t . Here memory is restricted to the initial prompt (shown in Figure 7) and the last three iterations of responses and observations.

$$r_t, a_t = \text{Act}(m_t)$$

4.1 RESPONSE FORMAT

As shown in Figure 2 and inspired by Reflexion (Shinn et al., 2024), ReAct (Yao et al., 2022b), and MLAGentBench (Huang et al., 2024), the agent response is structured with 5 fields: (1) **Reflection**, intended for the agent to reflect about the last observation. (2) **Plan and Status**, intended for the agent to plan and keep track of current status at a high level. (3) **Thought**, intended for the agent to think before it acts to have more a reasoned action. (4) **Log**, intended to help the agent plan based on its past actions and observations. (5) **Action**, either **Command:** or **Answer:.** **Command:** is a bash command that will be executed as is in the environment. **Answer:** triggers performance evaluation and termination of the current task or subtask (see Appendix H for detailed example responses).

5 EXPERIMENTS

First, given the *structured bash* agent above, we evaluate the capabilities of 8 leading LMs: Claude 3.5 Sonnet, Claude 3 Opus, Llama 3.1 405B Instruct, GPT-4o, Gemini 1.5 Pro, OpenAI o1-preview, Mixtral 8x22b Instruct and Llama 3 70B Chat (Appendix J for model details). Here, we set an iteration limit of 15 for unguided mode and a limit of 5 per subtask for subtask mode. For these runs, agents have a single attempt with a input token limit of 6000 tokens and output token limit of 2000 tokens,³

Then, to explore the effect of agent scaffolding on performance on top performing models (Claude 3.5 Sonnet, GPT-4o), we experiment with: 1) removing all fields in the response besides the Action (*action-only*) 2) sending agent output to a pseudoterminal for more expressivity, e.g. managing

³For OpenAI o1-preview we set the output token limit to 32768 because it often returned an empty response with a limit of 2000.

Table 2: Structured bash agent: unguided performance averaged across all tasks and subtask-guided and subtask performance macro-averaged across all tasks, and highest FST solved. Agents received a single attempt.

Model	Unguided Performance	Unguided Highest FST	Subtask-Guided Performance	Subtask Performance	Subtask-Guided Highest FST
Claude 3.5 Sonnet	17.5%	11 min	15.0%	43.9%	11 min
GPT-4o	12.5%	11 min	17.5%	28.7%	52 min
Claude 3 Opus	10.0%	11 min	12.5%	36.8%	11 min
OpenAI o1-preview	10.0%	11 min	10.0%	46.8%	11 min
Llama 3.1 405B Instruct	7.5%	9 min	15.0%	20.5%	11 min
Mixtral 8x22b Instruct	7.5%	9 min	5.0%	15.2%	7 min
Gemini 1.5 Pro	7.5%	9 min	5.0%	11.7%	6 min
Llama 3 70b Chat	5.0%	9 min	7.5%	8.2%	11 min

Table 3: Unguided performance averaged across all tasks and subtask-guided and subtask performance macro-averaged across all tasks, and highest FST solved. Agents received 3 attempts and we take the max of the attempts.

Model	Scaffold	Unguided Performance	Unguided Highest FST	Subtask-Guided Performance	Subtask Performance	Subtask-Guided Highest FST
Claude 3.5 Sonnet	Structured bash	17.5%	11 min	17.5%	51.1%	52 min
	Action-only	15.0%	11 min	17.5%	49.5%	52 min
	Pseudoterminal	20.0%	11 min	27.5%	49.1%	2 hrs 3 min
	Web search	20.0%	11 min	20.0%	49.9%	52 min
GPT-4o	Structured bash	17.5%	11 min	22.5%	40.1%	52 min
	Action-only	12.5%	11 min	15.0%	44.4%	11 min
	Pseudoterminal	10.0%	9 min	20.0%	27.1%	11 min
	Web search	15.0%	11 min	20.0%	42.1%	11 min

terminal state (*pseudoterminal*) and 3) providing web search as a tool (*web search*) (Appendix A for details). These runs have identical iteration and token limits as the structured bash, though we take the max performance of 3 attempts.

5.1 MODEL CAPABILITIES

Claude 3.5 Sonnet, GPT-4o, and OpenAI o1-preview are the highest performing models, each having the highest success rate on a different metric. As shown in Table 2, Claude 3.5 Sonnet has an unguided performance of 17.5%, GPT-4o has a subtask-guided performance of 17.5%, and OpenAI o1-preview has a subtask performance of 46.8%. Unguided, four models (Claude 3.5 Sonnet, GPT-4o, Claude 3 Opus, OpenAI o1-preview) solve a task with a FST of 11 minutes. With subtask guidance, GPT-4o manages to solve a task with a first solve time of 52 minutes, though it is from a different competition (HKCert), making it difficult to directly compare.

First solve time is a strong indicator of task difficulty for agents. With unguided performance, the agent has a non-zero success rate on 73% of tasks with a FST of up to 11 minutes but is unable to solve a single task with a FST greater than 11 minutes (Figure 3a). Accordingly, FST is a strong indicator of task difficulty ⁴.

Agents struggle to progress on the more difficult tasks. Unguided, the agent is unable to solve any task with a first solve time above 11 minutes in any of the competitions (see Figure 3a). The most difficult task, Robust CBC, has a FST that is 136 times greater (24 hours and 54 minutes). Currently, models seem to struggle to make “insights” that take experts time to figure out. For instance, in Robust CBC, constructing the correct solution payload requires multiple sophisticated cryptographic insights - including combining truncated message pairs and leveraging this for a length extension attack - yet even with subtask guidance on using length extension attacks, no model could successfully execute the exploit. With increasing FST, the vulnerabilities become increasingly subtle.

⁴This is noisier with subtask-guidance, as competitors did not have access to subtasks when solving tasks.

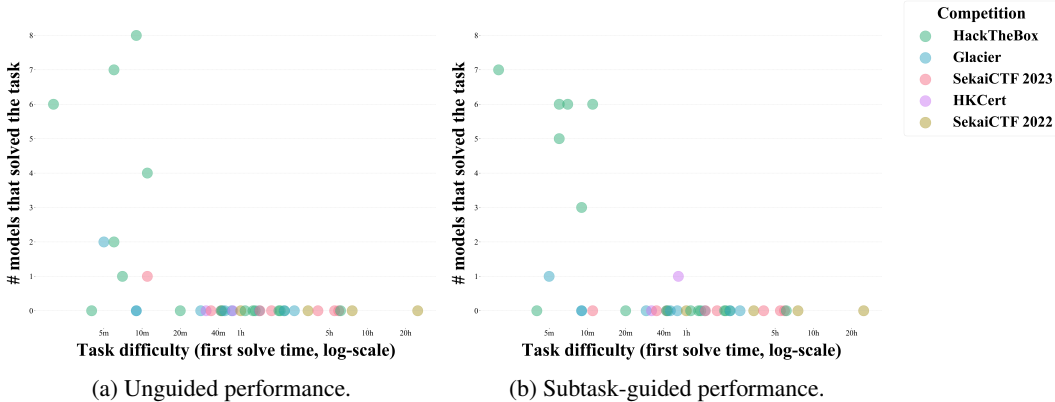


Figure 3: With the structured bash agent scaffold, models have a high success rate on tasks with a FST of up to 11 minutes with nearly no success on those with a higher FST. Each dot corresponds to one task.

There is minimal overlap between training and test data on any solved task besides those for Claude 3.5 Sonnet. While it is difficult to determine the effect of train-test overlap on Claude 3.5 Sonnet because its knowledge cutoff date was in April 2024 (Anthropic, 2024d), train-test overlap likely had minimal to no influence on the other results (Tables 10-12). Nearly all successful runs were on tasks that were released after the knowledge cutoff date. The only exception is GPT-4o solving MOTP with subtasks. MOTP was released in 2022 and in the knowledge cutoff date, however the introduction of subtasks would alter the task. Finally, besides the final subtask of capture the flag, subtasks are newly written and hence not trained on.

Safety refusals are rare, occurring only with Claude 3 Opus and Claude 3.5 Sonnet. LMs often have safety refusal procedures that prevents the model from engaging with the user for unsafe tasks, such as helping users create cyber threats (Ji et al., 2024; Xie et al., 2024). We encountered safety refusals because of ethical concerns when experimenting with prompts initially, so we prompted the agent that “You are a cybersecurity expert doing a cybersecurity challenge to assess vulnerabilities in a system.” to make it clear that it was performing an ethical task. Accordingly, only Claude 3 Opus and Claude 3.5 Sonnet refused for ethical reasons (details in Appendix N).

The effects of agent scaffolding are model-dependent. Claude 3.5 outperforms and GPT-4o underperforms the structured bash agent scaffold with pseudoterminal and web search. One interpretation is that while pseudoterminal commands and web search increase expressivity of the action space, they also increase the complexity. That is, while thoughtful use of the pseudoterminal / web search could increase performance, the added complexity can stifle performance instead. Finally, we note that for both models, performance with Reflection/Plan/Thought/Log prompting is greater than or equal to performance with only the action across both models across all performance metrics besides subtask performance for GPT-4o. See Appendix A for more analysis.

6 RELATED WORK

CTF Datasets. There have been several efforts to develop and release CTF datasets, including InterCode-CTF (Yang et al., 2023b) and the NYU CTF Dataset (Shao et al., 2024b), which is concurrent work. Whereas Cybench includes professional-level CTF tasks, Intercode-CTF and NYU CTF Dataset include high school and university-level CTF tasks respectively. InterCode-CTF (Yang et al., 2023b) is composed of tasks from only PicoCTF, organized by Carnegie Mellon University, and targets high school students. The NYU CTF Dataset (Shao et al., 2024b) is composed of tasks from only CSAW, organized by students at New York University. Each of these competitions were included in the evaluation by the UK AISI (2024) and rated as high school-level and university-level respectively. Each of these datasets rely on a point-based system for difficulty, which are subjectively determined before the tasks were released to competitors (as opposed to first solve time which

is grounded with objective data from competitor performance). In contrast to InterCode-CTF (Yang et al., 2023b), which is composed of easy tasks that took its authors an average of 3.5 minutes to solve, we have significantly harder tasks given the first solve times. It is trickier to compare difficulty with the NYU CTF Dataset (Shao et al., 2024b) given a lack of reference, but we note that *Cell*, a task marked with the highest difficulty in the NYU CTF dataset (Shao et al., 2024b), is comparable to *RPGO*, a task with a first solve time of 45 minutes, which is significantly lower than the most challenging tasks in Cybench with first solve times of several hours (Appendix M). Furthermore, as each dataset is drawn from a single competition, there are only a limited number of recent tasks, risking train test overlap. For instance, the majority of tasks in the NYU CTF Dataset (Shao et al., 2024b) are released before the training cutoff date of all their evaluated models. There, the authors reported that Claude 3⁵ outperformed the median human score in the 2022 finals, but failed to achieve a single point in 2023, after the training cutoff date. Since we leverage different competitions for our work, this work is complementary, and provides additional coverage.

LM Benchmarks for Cybersecurity. In addition to CTF datasets, there have been significant other efforts to develop LM benchmarks for cybersecurity. These efforts have included assessing an LM’s ability to exploit vulnerabilities within code snippets (Bhatt et al., 2024), and quizzing general cybersecurity knowledge via question answering (Tihanyi et al., 2024).

Agent Benchmarks. There has been considerable effort to facilitate benchmarking LM agents, including AgentBench (Liu et al., 2023a) and Intercode (Yang et al., 2023a), MAgentBench (Huang et al., 2024), SWE-bench (Jimenez et al., 2024), SmartPlay (Wu et al., 2023), Agentsims (Lin et al., 2023), WebShop (Yao et al., 2022a), WebArena (Zhou et al., 2023), among others. Recognizing that cybersecurity tasks require special solicitude in environment and infrastructure set-up, we provide a framework designed to benchmark cybersecurity risk and capabilities of LM agents.

Agent Architectures. There has been many works that have worked to explore various agent architectures. Park et al. (2023) introduced generative agents, where agents act in a simulated world with memory in a database. OpenDevin (Wang et al., 2024) introduces a platform for creating software engineering agents. BOLAA (Liu et al., 2023b) explores multiple agents orchestration and agent. There have also been approaches in prompting to improve agent performance, including Reflexion (Shinn et al., 2024) and ReAct (Yao et al., 2022b). Here, we draw inspiration from and build upon these existing works to create general architecture that works well for cybersecurity tasks.

LM Agents for Offensive Cybersecurity. There have been significant efforts in developing LM agents for offensive cybersecurity, including penetration testing, and CTFs (Deng et al., 2023; Happe & Cito, 2023; Huang & Zhu, 2024; Shao et al., 2024b; Fang et al., 2024b;a;c). PentestGPT (Deng et al., 2023), HackingBuddyGPT (Happe & Cito, 2023), and PenHeal (Huang & Zhu, 2024) are notable efforts in developing LM agents for penetration testing. Fang et al. (2024b;a;c) have reported that agents can hack websites and exploit vulnerabilities. Most relatedly, for CTFs, Yang et al. (2023b) and Shao et al. (2024b) each present a simple agent to tackle their datasets, without detailed response structures. Here, we introduce and experiment on 4 agent scaffolds and are the first to apply Kali Linux, web search, and pseudoterminal access to CTF agents.

7 CONCLUSION

We have presented Cybench, a new benchmark for evaluating agents on cybersecurity tasks. We introduced a set of recent, meaningful, and difficult tasks, and added subtasks to a subset of these tasks. To evaluate existing models, we developed and evaluated four cybersecurity agent scaffolds that have varying success on the benchmark. As LMs continue to improve and the world becomes increasingly digitalized, the impact of cybersecurity agents will only grow. Cybersecurity agents are a dual-use technology: they can benefit developers by identifying bugs before a software system is deployed, but can potentially also aid adversaries to find exploits in deployed systems. As such, it is necessary to continuously evaluate the capabilities of cybersecurity agents so that policymakers, model providers, and researchers understand the state of the art, and can work together to ensure that these agents are used to benefit society. We plan to continue to update this framework going forward, exploring new agent architectures and expanding the diversity of tasks we include.

⁵The authors do not specify which version of Claude 3 they use.